denominator becomes equal to `k-1`, and it diminishes of `1` unit as `i` increases until, for `i=k-1`, it becomes equal to `2`.

Similar considerations can be made for the last term of the formula, giving us:

$$E[D_k] = \sum_{j=2}^{k-1} \frac{1}{j} + \sum_{j=2}^{n-k} \frac{1}{j} = \sum_{j=1}^{k-1} \frac{1}{j} - 1 + \sum_{j=1}^{n-k} \frac{1}{j} - 1 = H_{k-1} - 1 + H_{n-k} - 1$$

The two summations in the formula above, in fact, are both partial sums of the harmonic series, denoted as `Hn`; `Hn<ln(n)`, we finally obtain:

$$E[D_k] = H_{k-1} + H_{n-k} - 2 < ln(k-1) + ln(n-k) - 2 < 2 \cdot ln(n) - 2$$

This result guarantees that, over a large number of attempts, the mean value for the height of a randomized treap is `O(n)`, logarithmic in the number of keys stored (independently of the order keys are added or removed, and on the distribution of keys).

3.5.2 Profiling Height

So far for theory: moreover, you could object, it's just a guarantee on the average among several attempts, what if we get really unlucky on a crucial run?

To get a better insight on the actual performance of this data structure, we can run a little profiling, just like we did in section 2.10 for d-heaps.

This time, we'll use a profiling tool for Java, JProfiler[12].

We could even omit using a profiling tool, because in our tests we are going to compare an implementation of a plain BST versus a